Endpoint Security Optimization with Microsoft Intune

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — optimization: Endpoint Security with Microsoft Intune 5
- What you get . . . 5
- What this delivers . . . 5
- The standard deployment model . . . 5
- Boundary . . . 6
- Effort . . . 6

1. Solution Overview — optimization: Endpoint Security with Microsoft Intune 6
- What this solution is . . . 6
- How the technology works (the levers) . . . 6
- The standard deployment model . . . 7
- Scope (what this solution changes) . . . 7
- Out of scope . . . 7
- Connections and authentication used . . . 8
- Limitations (be honest with buyers) . . . 8

2. Requirements Guide — optimization: Endpoint Security with Microsoft Intune 8
- 2.1 An existing, working Intune endpoint-security deployment . . . 8
- 2.2 A defined pilot device ring . . . 9
- 2.3 Current security-baseline version awareness . . . 9
- 2.4 Rights to change endpoint-security and compliance policy . . . 9
- 2.5 A before-state baseline (conflicts, compliance, coverage) . . . 9
- 2.6 Conditional Access impact understood before enforcing . . . 10
- 2.7 Change control, pilot scope, and rollback . . . 10
- 2.8 Response/enforcement governance . . . 10
- Checklist . . . 10

Global Requirements — Defender (category-level) 10
- G1. Microsoft 365 / Entra tenant . . . 11
- G2. Defender licensing . . . 11
- G3. Administrative roles (least privilege) . . . 11
- G4. Onboarding reach to endpoints / identity / workloads . . . 11
- G5. Network access to Defender service endpoints . . . 11
- G6. A pilot ring . . . 11
- G7. Log Analytics workspace (where required) . . . 12
- G8. Response governance . . . 12

3. Implementation Guide — optimization: Endpoint Security with Microsoft Intune 12
- Naming (set once, reuse verbatim) . . . 12

Step 1 — Baseline the current estate (measure "before") . . . 12
Step 2 — Migrate security baselines to the current version (pilot) . . . 13
Step 3 — Resolve policy conflicts (pilot) . . . 13
Step 4 — Tune compliance and understand the Conditional Access impact (pilot) . . . 13
Step 5 — Expand coverage to unassigned devices (pilot → estate) . . . 14
Step 6 — Validate the pilot (measure "after") . . . 14
Step 7 — Expand ring by ring across the estate . . . 14
5. Testing Guide — optimization: Endpoint Security with Microsoft Intune 15
Test 1 — Before-state baseline captured . . . 15
Test 2 — Latest security-baseline version applied on the pilot ring . . . 15
Test 3 — Conflicts/errors reduced on the pilot ring . . . 15
Test 4 — Compliance false-positives reduced, real gaps still caught . . . 15
Test 5 — Conditional Access impact modeled before enforce . . . 15
Test 6 — Coverage expanded . . . 16
Test 7 — No new conflicts introduced by expansion . . . 16
Test 8 — Policies propagate to pilot devices . . . 16
6. Troubleshooting Guide — optimization: Endpoint Security with Microsoft Intune 16
No baseline / report empty . . . 16
Baseline migration not applying . . . 16
Conflict persists after consolidation . . . 17
Compliance tuned too loose/tight . . . 17
Unexpected Conditional Access blocks . . . 17
Coverage gap / double assignment . . . 17
Policy stuck pending / not applying . . . 17
Script cannot authenticate or read Intune data . . . 17
7. Best Practices — optimization: Endpoint Security with Microsoft Intune 18
Always baseline before you touch anything . . . 18
One authoritative policy per setting . . . 18
Migrate baselines deliberately, keep only justified customizations . . . 18
Tune compliance for signal, not silence . . . 18
Treat Conditional Access as the blast radius . . . 18
Expand coverage as deliberately as you tune . . . 19
Keep licensing and preview facts current . . . 19
Defer the policy decisions to the owners . . . 19
Where to go next . . . 19
8. Deliverables Summary — optimization: Endpoint Security with Microsoft Intune 19

Documentation 19
PowerShell scripts (Microsoft Graph / Intune) 20
What the buyer receives 20
Outcome 21
Appendix A — PowerShell Scripts 22

Executive Summary — optimization: Endpoint Security with Microsoft Intune

White-label · Defender

An endpoint-security estate managed through **Microsoft Intune** works on day one and then **drifts**: security baselines fall behind Microsoft's updated versions, multiple policies **conflict** so devices land in an unpredictable state, **compliance policies** flag noise or miss real gaps, and new devices **never receive** the security policy at all. This solution is a **tuning engagement, not a re-deployment** — it takes an existing Intune endpoint-security configuration and makes it **measurably better**: it **tunes the security baselines and compliance policies**, **finds and resolves policy conflicts**, **expands coverage** to unassigned devices, and **aligns settings to current best practice** — all **pilot-first, measured with before/after metrics, and reversible.**

This is a complete, **do-it-today** package written for an administrator with no prior optimization experience. Every change is proven on a **pilot device ring** first, disruptive compliance changes start in a **non-impactful / staged** posture, and every step names a **rollback** so you improve security posture **without locking users out or breaking production.**

What you get

• A plain-language **Solution Overview** of what "optimizing" an Intune endpoint-security estate means — baseline tuning, conflict resolution, compliance tuning, coverage expansion, and best-practice alignment — and how it differs from the read-only assessment.

• A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (this is a *tuning* engagement, so the prerequisite is a **working** Intune endpoint-security deployment, not a green field).

• An **Implementation Guide** of exact, ordered portal steps — baseline the current state, migrate to the latest security-baseline version, resolve conflicts, tune compliance and Conditional Access impact, expand assignments, and align to best practice — then validate the deltas and expand.

• Five **PowerShell scripts** (Microsoft Graph / Intune) for read-only discovery, a per-device-and-setting **conflict** report, a `-WhatIf`-gated assignment-expansion helper, validation, and a **before/after** metrics report.

• **Testing** and **Troubleshooting** guides for a confident, low-drama rollout.

What this delivers

An Intune endpoint-security estate with **fewer conflicts** (a single authoritative policy per setting, error/conflict counts down), **broader coverage** (previously unassigned devices now receiving baseline, compliance, and Defender policy), **tuned compliance** (fewer false non-compliant flags, real gaps caught, Conditional Access impact understood), and **current baselines** (the latest security-baseline version applied) — every change **piloted, measured against a captured baseline, and reversible**.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — baseline and tune on a pilot device ring, measure the before/after delta, then expand across the estate.

Boundary

This solution delivers the **technical tuning and hardening configuration** and the **before/after evidence**. Which compliance thresholds the organization requires, whether a Conditional Access grant should block or warn, and what constitutes acceptable residual risk are **the security/identity team's determinations** — this guide provides the controls and the measurements, not the policy decision. Conditional Access changes stay **report-only / staged first**. Plan/license/preview facts (Intune plan, security-baseline versions, Defender for Endpoint P2 features surfaced through Intune) are flagged **verify current availability in your tenant**.

Effort

A competent administrator can baseline the estate and complete a first tuning pass — baseline-version migration, conflict resolution, a first compliance-tuning round, coverage expansion, and best-practice alignment — in a focused engagement on a pilot ring, then expand ring by ring. Ongoing, the tuning cadence is a recurring hour per cycle to keep conflicts at zero and coverage complete as baselines update.

1. Solution Overview — optimization: Endpoint Security with Microsoft Intune

What this solution is

This is a **optimization** — a structured tuning engagement for an **existing, working** Microsoft Intune endpoint-security estate. It is deliberately **distinct from the read-only assessment**: the assessment *tells you* what to improve; this solution *applies* the improvements, pilot-first, with **before/after measurement** and a **rollback** for every change. If you have not yet deployed endpoint security through Intune, this is the wrong solution — deploy first (see *Where to go next* in Best Practices), then optimize.

It focuses on the four highest-value tuning levers on a mature Intune endpoint-security deployment:

1. **Baseline & policy tuning** — migrate **security baselines** to the current version, reconcile customized settings, and align antivirus/ASR/EDR/firewall/disk-encryption policy to current best practice.
2. **Conflict reduction** — find where **two or more policies set the same setting differently** (the cause of "conflict"/"error" states) and consolidate to a single authoritative policy per setting.
3. **Compliance tuning** — reduce false **non-compliant** flags, catch the real gaps, and understand the **Conditional Access** impact before tightening.
4. **Coverage expansion** — find devices and users **not assigned** baseline, compliance, or Defender policy, and bring them into scope with a consistent group/assignment model.

How the technology works (the levers)

• **Security baselines** are Microsoft-published, versioned bundles of recommended settings (Security Baseline for Windows, Defender for Endpoint baseline, Edge baseline). New versions ship regularly; devices on an **old baseline version** drift from current guidance. Migrating means creating the new-version baseline, carrying forward justified customizations, and retiring the old one.

• **Policy conflict** happens when a device is targeted by **multiple** configuration/endpoint-security/baseline policies that set the **same** setting to **different** values. Intune reports the device/setting as **Conflict** or **Error**, and the effective value is undefined. The fix is to make **one** policy authoritative for each setting and remove the overlap.

• **Compliance policies** evaluate device state (encryption, OS version, Defender health, risk level) and mark devices **compliant / non-compliant**; **Conditional Access** then grants or blocks access based on that. Tuning means the thresholds reflect reality (so you don't flag benign devices) while still catching genuine gaps — and understanding the CA blast radius before enforcing.

• **Assignments** target policies to **Entra groups** (device or user). A device only gets a control if a policy is **assigned** to a group it belongs to — coverage gaps come from unassigned devices, empty dynamic groups, or exclusions that swallowed too much.

The standard deployment model

> ***Assess (baseline the current estate) → Configure (stage the tuning: new baseline version, conflict consolidation, compliance thresholds, assignment model) → Pilot (apply to a device ring) → Validate (measure the before/after delta) → Expand (ring by ring across the estate).***
> *Every disruptive change (especially compliance/Conditional Access) starts **report-only / staged**, and every step names a rollback.*

Scope (what this solution changes)

• Migrates **security baselines** to the current version and reconciles customized settings.

• **Resolves policy conflicts** by consolidating to a single authoritative policy per setting.

• **Tunes compliance policies** (thresholds, grace periods, actions) and clarifies the Conditional Access impact.

• **Expands assignments** to unassigned/stragging devices with a consistent group model.

• Captures a **before/after metrics baseline** (conflict/error counts, non-compliant proportion, coverage %) so the improvement is measurable.

Out of scope

• **First-time Intune / endpoint-security deployment** — this tunes an existing deployment (deployment solution referenced in Best Practices).

• **Full device-configuration / MDM management** beyond endpoint security (app deployment, Wi-Fi/VPN profiles, Autopilot) — this focuses on the security policies.

• **Conditional Access design as a whole** — this tunes the compliance→CA impact of endpoint security; broader CA architecture is a separate identity engagement.

• **Regulatory, records/legal, and acceptable-risk determinations** — deferred to the customer's compliance/security/identity functions.

Connections and authentication used

• The **Microsoft Intune admin center** (`https://intune.microsoft.com`) → **Endpoint security** (Antivirus, Disk encryption, Firewall, EDR, ASR, Account protection, Security baselines), **Devices** → **Compliance**, and **Groups**.

• The **Microsoft Entra admin center** (`https://entra.microsoft.com`) → **Conditional Access** (to review the compliance grant impact) and **Groups**.

• **Microsoft Graph PowerShell** (`Microsoft.Graph.Authentication` and DeviceManagement modules) for read-only discovery, conflict reporting, validation, before/after metrics, and the `-WhatIf`-gated assignment helper — least-privilege scopes such as `DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`, and `Group.Read.All`.

• Least-privilege roles: **Intune Read Only Operator** / **Security Reader** for read-only work; **Endpoint Security Manager** (and **Intune Administrator** where required) for applying changes.

Limitations (be honest with buyers)

• Some capabilities are **plan-gated**: Intune requires the appropriate plan/license per user; Defender-for-Endpoint settings surfaced through Intune (ASR/EDR reporting) need **Defender for Endpoint P2**. *Verify current plan requirements and availability in your tenant.*

• Policy changes **propagate on the device check-in cadence**, not instantly — allow sync time before judging a change.

• Resolving conflicts requires deciding **which** policy is authoritative — that is a design decision this solution structures but the security team owns.

• Compliance tuning changes **who can access what** via Conditional Access; that is why CA impact is reviewed **report-only first** and the identity team owns the enforce decision.

2. Requirements Guide — optimization: Endpoint Security with Microsoft Intune

Verify **every** requirement before tuning. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, Defender licensing, admin roles, onboarding reach, network access, pilot ring, workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to optimizing an existing Intune endpoint-security deployment**. Because this is a **tuning** engagement, the defining prerequisite is that Intune endpoint security is **already deployed and applying** — not a green field. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 An existing, working Intune endpoint-security deployment

Description: Intune already manages devices with **endpoint-security policies** (baselines, antivirus, disk encryption, firewall, EDR/ASR) and **compliance policies** applied — not a new deployment. **Why:** This solution *tunes* an existing estate; there must be enrolled devices, applied policies, and compliance results to optimize. **Verify:** `intune.microsoft.com` → **Endpoint security** shows policies with device-assignment results, and **Devices** → **Compliance** shows evaluated devices; run `SOPEIS-Discovery.ps1`. **Implement-if-missing:** If endpoint security is not deployed through Intune, stop — deploy it first, then return to optimize.

2.2 A defined pilot device ring

Description: A named Entra group of representative pilot devices to tune first: `Intune Optimize – Pilot Ring – Pilot`. **Why:** Baseline migration, conflict consolidation, and compliance/CA changes are validated on a ring before estate-wide rollout (per `global-requirements-defender.md` G6). **Verify:** The Entra group exists and contains a representative sample (one of each OS/build/role); `SOPEIS-Discovery.ps1` lists groups and their device counts. **Implement-if-missing:** Create the pilot group in **Entra** → **Groups** (dynamic or assigned) before applying any change.

2.3 Current security-baseline version awareness

Description: Knowledge of which **security-baseline version(s)** are currently applied versus the latest version Microsoft publishes. **Why:** Devices on an older baseline version drift from current guidance; migration is a core tuning lever, and the version must be known to plan it. **Verify:** `intune.microsoft.com` → **Endpoint security** → **Security baselines**; note each baseline's version and whether a newer one is available. *Baseline versions change — verify current.* **Implement-if-missing:** Record the applied versions and the latest available (Implementation Step 2 migrates them).

2.4 Rights to change endpoint-security and compliance policy

Description: Endpoint Security Manager (and **Intune Administrator** where full policy authoring/assignment is needed), plus **Intune Read Only Operator** / **Security Reader** for reporting accounts. Reviewing Conditional Access impact needs at least **Security Reader** in Entra. **Why:** Editing baselines, resolving conflicts, tuning compliance, and changing assignments require write access; reporting and CA review only need read. **Verify:** Intune → **Tenant admin** → **Roles**; Entra → **Roles and administrators**; `SOPEIS-Discovery.ps1` reports the running account's effective scopes. **Implement-if-missing:** Assign the minimum roles at the narrowest scope; prefer PIM eligible activation over standing access.

2.5 A before-state baseline (conflicts, compliance, coverage)

Description: A saved snapshot of current **conflict/error counts**, **non-compliant proportion**, and **coverage** (devices receiving each policy type) — the "before" for before/after measurement. **Why:** Optimization is only credible if the improvement is **measured**; the baseline is the reference every later report compares against. **Verify:** A baseline file exists; run `SOPEIS-Report.ps1 -SaveBaseline`. **Implement-if-missing:** Run `SOPEIS-Report.ps1 -SaveBaseline` before making any change (Implementation Step 1).

2.6 Conditional Access impact understood before enforcing

Description: A read of the **Conditional Access** policies that grant/block on device compliance, so tightening compliance doesn't unexpectedly lock users out. **Why:** Compliance state feeds Conditional Access; a stricter compliance policy can turn a benign device non-compliant and block sign-in. The blast radius must be understood first. **Verify:** `entra.microsoft.com` → **Conditional Access**; identify policies with a **Require device to be marked compliant** grant. Use **Report-only** / the What-If tool. *CA changes are the identity team's decision.* **Implement-if-missing:** Put new/stricter CA grants in **Report-only** first; model impact before enforcing (Implementation Step 4).

2.7 Change control, pilot scope, and rollback

Description: An approved change record, the named pilot ring, and a documented rollback before migrating baselines, consolidating conflicts, or tightening compliance. **Why:** These changes affect device configuration and user access; the pilot-first, report-only-first approach and a written back-out are the safety net. **Verify:** An approved change record, the pilot scope, and a written rollback (revert to the prior baseline/policy, restore prior assignment) exist. **Implement-if-missing:** Capture the baseline (2.5), define the pilot ring (2.2), and write the rollback before Step 2.

2.8 Response/enforcement governance

Description: Confirmation of the agreed posture for **compliance actions** (mark non-compliant immediately vs after a grace period; notify vs restrict) and that disruptive enforcement stays staged. **Why:** Compliance actions can restrict access; the organization must decide the guardrails and grace periods (per `global-requirements-defender.md` G8). **Verify:** A documented compliance-action posture exists; new stricter actions are staged with a grace period first. **Implement-if-missing:** Define the compliance-action posture and grace periods with the security/identity team. *Enforcement decisions are theirs; this guide covers the technical configuration.*

Checklist

- Existing, applying Intune endpoint-security + compliance deployment — `SOPEIS-Discovery.ps1`
- Pilot device ring `Intune Optimize – Pilot Ring – Pilot` defined
- Current vs latest security-baseline versions known
- Endpoint Security Manager (+ Intune Admin) and read-only/Security Reader roles
- Before-state baseline saved — `SOPEIS-Report.ps1 -SaveBaseline`
- Conditional Access compliance impact reviewed (report-only first)
- Change window, pilot scope, and rollback documented
- Compliance-action posture / grace periods agreed

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — optimization: Endpoint Security with Microsoft Intune

Follow these steps **in order**, in the **Microsoft Intune admin center** (`https://intune.microsoft.com`) and the **Microsoft Entra admin center** (`https://entra.microsoft.com`) for Conditional Access review. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Baseline the current estate, tune on the* ***pilot ring*** `Intune Optimize – Pilot Ring – Pilot` *first,* ***measure the before/after delta****, then expand ring by ring. Every disruptive change — especially compliance and Conditional Access — starts* ***report-only / staged****, and every step names a rollback.* ***Which compliance thresholds and Conditional Access grants are required is the security/identity team's decision.***

Naming (set once, reuse verbatim)

- Pilot device group (Entra): `Intune Optimize – Pilot Ring – Pilot`
- Migrated security baseline (pilot): `Security Baseline – Pilot Ring – Pilot`
- Migrated security baseline (production, on expand): `Security Baseline – All Devices – Prod`
- Consolidated endpoint-security policy (per setting area): `Endpoint Security – <area> – Prod` (e.g. `Endpoint Security – Antivirus – Prod`)
- Tuned compliance policy: `Compliance – Windows Corporate – Prod`
- Conditional Access (report-only during tuning): `CA – Require Compliant Device – ReportOnly`

Step 1 — Baseline the current estate (measure "before")

1. `intune.microsoft.com` → **Endpoint security** and **Devices** → **Compliance**. Note the current baseline versions, the count of devices in **Conflict/Error**, and the **non-compliant** proportion.

2. Run `SOPEIS-Discovery.ps1` to capture, read-only: all endpoint-security and compliance policies, their assignments, applied security-baseline versions, per-policy assignment coverage, and groups.

3. Run `SOPEIS-ConflictReport.ps1` to list devices/settings reporting **Conflict** or **Error** and the policies contributing to each.

4. Run `SOPEIS-Report.ps1 -SaveBaseline` to record the **before** metrics — conflict/error counts, non-compliant proportion, coverage % per policy type — to `sopeis-baseline.json`.

Step 2 — Migrate security baselines to the current version (pilot)

1. **Endpoint security** → **Security baselines** → open the baseline with a newer version available → **Create profile** on the **latest version**, named `Security Baseline – Pilot Ring – Pilot`.

2. Carry forward only **justified** customizations from the old profile; take Microsoft's new defaults otherwise. *Baseline versions/settings change — verify current.*

3. **Assign** the new baseline to `Intune Optimize – Pilot Ring – Pilot` **only**, and **remove the pilot ring** from the old baseline's assignment so the two don't overlap (overlap causes conflict — Step 3).

4. Force a sync on a pilot device; allow the check-in cadence to apply the new baseline.

Step 3 — Resolve policy conflicts (pilot)

1. Re-run `SOPEIS-ConflictReport.ps1` (scoped to the pilot ring). For each conflicting setting, identify every policy that sets it.

2. Choose **one authoritative** policy per setting area, named `Endpoint Security – <area> – Prod`. **Remove the setting** from the non-authoritative policies (or remove those policies' assignment to the ring) so only one source remains.

3. Prefer consolidating overlapping legacy device-configuration profiles into the security-baseline / endpoint-security policy rather than maintaining parallel copies.

4. Re-run `SOPEIS-ConflictReport.ps1` → confirm the conflict/error count on the pilot ring drops toward zero.

Step 4 — Tune compliance and understand the Conditional Access impact (pilot)

1. **Devices** → **Compliance** → **Policies** → open/clone the policy as `Compliance – Windows Corporate – Prod`. Adjust the settings that generate **false** non-compliance (e.g. an OS-version floor set too high, a Defender-signal check that's flaky) and ensure real gaps (encryption off, no Defender) still fail.

2. Set **Actions for noncompliance** with a **grace period** (notify before mark non-compliant) so users aren't blocked instantly. *The grace period and thresholds are the security team's decision.*

3. **Review the CA impact first:** `entra.microsoft.com` → **Conditional Access** → put/keep the compliance-grant policy in **Report-only** (`CA – Require Compliant Device – ReportOnly`) and use the **What-If** tool to model who would be blocked before enforcing. **The enforce decision is the identity team's.**

4. **Assign** the tuned compliance policy to the pilot ring; monitor results before expanding.

Step 5 — Expand coverage to unassigned devices (pilot → estate)

1. Run `SOPEIS-Discovery.ps1` (and review the coverage section) to find devices/users **not receiving** baseline, compliance, or Defender policy — the coverage gaps.

2. Fix the assignment model: point policies at a consistent set of groups; correct empty dynamic-group rules and over-broad **exclusions**.

3. **Preview the assignment change first:** run `SOPEIS-Set-Assignment.ps1 -PolicyName '<policy>' -GroupName 'Intune Optimize – Pilot Ring – Pilot'` (defaults to `-WhatIf`) to see the assignment it would add; re-run with `-Apply`.

4. Run `SOPEIS-Validate.ps1` → confirms coverage improved and no device is now double-targeted (which would re-introduce conflict).

Step 6 — Validate the pilot (measure "after")

1. Run `SOPEIS-Validate.ps1` → confirms the latest baseline applied to the pilot, conflict/error count near zero, tuned compliance policy assigned, and coverage improved.

2. Run `SOPEIS-Report.ps1 -BaselinePath .\sopeis-baseline.json` → shows the **before/after delta**: fewer conflicts/errors, lower false-non-compliant proportion, higher coverage.

3. Confirm no pilot user was unexpectedly blocked (check the CA report-only results). Complete the **Testing Guide** before expanding.

Step 7 — Expand ring by ring across the estate

1. Clone the validated pilot artifacts to production scope: `Security Baseline – All Devices – Prod`, the consolidated `Endpoint Security – <area> – Prod` policies, and the tuned compliance policy, assigned to the next ring, then the estate.

2. Only after the report-only CA impact is acceptable to the identity team, move `CA – Require Compliant Device – ReportOnly` to **On** in a staged manner.

3. Re-baseline with `SOPEIS-Report.ps1 -SaveBaseline` after each wave and track drift — conflicts should stay near zero and coverage complete as new devices and baseline versions appear.

Order of operations

`Baseline (before) → migrate baseline version (pilot) → resolve conflicts → tune compliance + review CA (report-only) → expand coverage → validate the before/after delta → expand ring by ring (enforce CA last)`

5. Testing Guide — optimization: Endpoint Security with Microsoft Intune

Run these checks after tuning the pilot ring (Implementation Steps 2–6), and again after each expansion wave. Each test has a **pass/fail gate**; do not expand across the estate until the pilot passes. Objects are referenced by the exact names from the Implementation Guide, and the headline gates are **before/after deltas** against the saved baseline.

Test 1 — Before-state baseline captured

Procedure: Confirm `sopeis-baseline.json` exists from `SOPEIS-Report.ps1 -SaveBaseline` (Implementation Step 1). **Pass:** A baseline with conflict/error counts, non-compliant proportion, and coverage % exists. **Fail:** No baseline — re-run Step 1; without a "before" there is no measurable improvement. See Troubleshooting *No baseline / report empty*.

Test 2 — Latest security-baseline version applied on the pilot ring

Procedure: Run `SOPEIS-Validate.ps1`. In **Endpoint security** → **Security baselines**, confirm `Security Baseline – Pilot Ring – Pilot` is on the **latest** version and assigned to `Intune Optimize – Pilot Ring – Pilot`. **Pass:** The pilot ring runs the latest baseline version and the old baseline no longer targets the ring. **Fail:** Old version still applied, or both baselines target the ring — re-run Implementation Step 2; see Troubleshooting *Baseline migration not applying*.

Test 3 — Conflicts/errors reduced on the pilot ring

Procedure: Run `SOPEIS-ConflictReport.ps1` scoped to the pilot ring and compare the conflict/error count to the Test 1 baseline. **Pass:** Conflict/error count on the pilot ring is **measurably lower** (target: near zero) than baseline. **Fail:** Conflicts unchanged or higher — a setting is still set by more than one policy; see Troubleshooting *Conflict persists after consolidation*.

Test 4 — Compliance false-positives reduced, real gaps still caught

Procedure: Run `SOPEIS-Report.ps1 -BaselinePath .\sopeis-baseline.json`. Compare the non-compliant proportion to baseline, and spot-check that a genuinely non-compliant device (e.g. encryption off) still fails. **Pass:** False non-compliant flags are **lower** than baseline, and a real gap still marks the device non-compliant. **Fail:** No reduction, or a real gap now passes (over-loosened) — re-tune the compliance policy; see Troubleshooting *Compliance tuned too loose/tight*.

Test 5 — Conditional Access impact modeled before enforce

Procedure: In **Entra** → **Conditional Access**, confirm `CA – Require Compliant Device – ReportOnly` is **Report-only** and review its results / What-If for the pilot ring. **Pass:** The compliance grant is report-only and the modeled block list contains no unexpected users. **Fail:** CA is already enforcing, or the model shows unexpected blocks — keep it report-only and re-tune; see Troubleshooting *Unexpected Conditional Access blocks*.

Test 6 — Coverage expanded

Procedure: Run `SOPEIS-Validate.ps1` and review the coverage section. Compare devices receiving baseline/compliance/Defender policy to the baseline. **Pass:** Coverage % per policy type is **higher** than baseline; previously unassigned devices now receive policy. **Fail:** Coverage unchanged, or devices now double-targeted — confirm the assignment model; see Troubleshooting *Coverage gap / double assignment*.

Test 7 — No new conflicts introduced by expansion

Procedure: Run `SOPEIS-ConflictReport.ps1` after the assignment changes. **Pass:** Expanding coverage did **not** re-introduce conflicts (each setting still has one authoritative policy). **Fail:** New conflicts appeared — an added assignment overlaps an existing policy; see Troubleshooting *Conflict persists after consolidation*.

Test 8 — Policies propagate to pilot devices

Procedure: Run `SOPEIS-Validate.ps1` and, on a pilot device, force an Intune sync and confirm the baseline/compliance policy shows applied. **Pass:** Pilot devices report the tuned policies as applied (allowing check-in time). **Fail:** Devices stuck pending/not applied — see Troubleshooting *Policy stuck pending / not applying*.

Exit gate

All eight tests pass on the pilot ring — with a **measurable before/after improvement** in Test 3, Test 4, and Test 6 — **before** any estate-wide expansion (Implementation Step 7). Conditional Access is moved from report-only to enforce **last**, only after the identity team accepts the modeled impact. Re-run Tests 2–8 after each expansion wave and compare against the re-saved baseline.

6. Troubleshooting Guide — optimization: Endpoint Security with Microsoft Intune

Common field issues, in **symptom** → **cause** → **fix** form. Run `SOPEIS-Discovery.ps1` and `SOPEIS-ConflictReport.ps1` first — most causes surface there.

No baseline / report empty

Symptom: `SOPEIS-Report.ps1` returns no metrics, or there is no "before" to compare against. **Cause:** The baseline was never saved, or the account lacks read access to policies/compliance/managed devices. **Fix:** Run `SOPEIS-Report.ps1 -SaveBaseline` **before** making changes (Implementation Step 1); confirm the account has **Intune Read Only Operator** / **Security Reader** and the Graph scopes (`DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`, `Group.Read.All`). Re-run.

Baseline migration not applying

Symptom: Pilot devices stay on the old baseline version after Implementation Step 2. **Cause:** The new-version baseline is unassigned or targets the wrong group, the old baseline **still targets** the ring (overlap), or the device hasn't checked in. **Fix:** Confirm `Security Baseline – Pilot Ring –`

`Pilot` is on the latest version and assigned to `Intune Optimize – Pilot Ring – Pilot`; **remove the pilot ring from the old baseline's assignment**; force a sync and allow the check-in cadence. Re-run `SOPEIS-Validate.ps1`.

Conflict persists after consolidation

Symptom: `SOPEIS-ConflictReport.ps1` still shows a setting in Conflict after you consolidated. **Cause:** More than one policy still sets that setting (a legacy device-configuration profile, a second endpoint-security policy, or the old baseline), or an added assignment re-introduced overlap. **Fix:** Use the report to find **every** policy contributing the setting; remove the setting (or the assignment) from all but the authoritative `Endpoint Security – <area> – Prod` policy. Re-check after devices sync. Avoid double-targeting when expanding coverage (Test 7).

Compliance tuned too loose/tight

Symptom: After tuning, either benign devices are still flagged non-compliant, or a genuinely non-compliant device now passes. **Cause:** A threshold is still mis-set (too strict → false positives; too lax → real gap passes). **Fix:** Re-open `Compliance – Windows Corporate – Prod`; align the thresholds so real gaps (encryption off, Defender unhealthy) fail and benign variance passes; keep a grace period on the action. Re-run `SOPEIS-Report.ps1 -BaselinePath .\sopeis-baseline.json`. *Thresholds are the security team's decision.*

Unexpected Conditional Access blocks

Symptom: Users are (or would be) blocked after a compliance change. **Cause:** A Conditional Access **Require compliant device** grant is enforcing against devices the tuned compliance policy now marks non-compliant. **Fix:** Keep `CA – Require Compliant Device – ReportOnly` in **Report-only**; use the **What-If** tool to model impact; re-tune compliance so legitimate devices pass; move to enforce **last** and in stages. *The enforce decision is the identity team's.*

Coverage gap / double assignment

Symptom: Some devices receive no policy, or a device is now targeted by two policies. **Cause:** Unassigned policies, an empty dynamic-group rule, an over-broad exclusion, or an added assignment that overlaps an existing one. **Fix:** Fix the assignment model to a consistent group set; correct dynamic-group rules and exclusions; preview assignment changes with `SOPEIS-Set-Assignment.ps1 -WhatIf` before applying; verify no double-targeting with `SOPEIS-ConflictReport.ps1`. Re-run `SOPEIS-Validate.ps1`.

Policy stuck pending / not applying

Symptom: A pilot device shows the policy as **Pending** or **Not applicable** and never applies. **Cause:** The device hasn't checked in, isn't a member of the assigned group, or the policy's OS/platform filter excludes it. **Fix:** Force an Intune sync on the device; confirm group membership and that the platform/filter matches the device; allow the check-in cadence. Re-check with `SOPEIS-Validate.ps1`.

Script cannot authenticate or read Intune data

Symptom: `SOPEIS-*.ps1` fails on connect or returns nothing. **Cause:** `Microsoft.Graph.Authentication` is missing, the account isn't signed in, or it lacks the required read scopes/consent. **Fix:** Let the script install the module (it checks on run); `Connect-MgGraph` with the listed scopes and consent them; confirm at least Intune Read Only Operator / Security Reader. Re-run the script.

7. Best Practices — optimization: Endpoint Security with Microsoft Intune

Field-proven guidance for tuning an Intune endpoint-security estate so it gets **measurably better** without locking users out or breaking production.

Always baseline before you touch anything

- Capture the **before** state (`SOPEIS-Report.ps1 -SaveBaseline` and `SOPEIS-ConflictReport.ps1`) first. Optimization you cannot measure is just change; the baseline is what makes "better" provable to leadership.
- Re-baseline after each expansion wave so the trend reflects real scope growth, not just the pilot.

One authoritative policy per setting

- **Conflict is the number-one drift symptom.** A setting configured by two policies has an undefined effective value. Consolidate to a single authoritative `Endpoint Security – <area> – Prod` policy per setting area and remove the overlap — don't maintain parallel copies.
- Prefer folding legacy device-configuration profiles into the security baseline / endpoint-security policy rather than running both.

Migrate baselines deliberately, keep only justified customizations

- When a new **security-baseline version** ships, create the new-version profile, carry forward only the customizations with a documented reason, and take Microsoft's new defaults otherwise. Then **retire the old baseline's assignment** so the two don't overlap (which would re-create conflict).

Tune compliance for signal, not silence

- Reduce **false** non-compliant flags so the queue is trustworthy, but never loosen a policy so far that a real gap (encryption off, Defender unhealthy) passes. The goal is that "non-compliant" means something.
- Give users a **grace period** before a compliance action restricts access — instant blocking generates help-desk load and erodes trust.

Treat Conditional Access as the blast radius

- Compliance state feeds Conditional Access. Always model a stricter compliance policy's impact **in report-only** (and with the What-If tool) before enforcing, and move to enforce **last** and in stages. **The**

enforce decision is the identity team's.

Expand coverage as deliberately as you tune

• A device that never received the policy is silently unprotected. Hunt for unassigned devices every cycle and fix the assignment model — but verify you didn't **double-target** a device, which re-introduces conflict.

• Use a consistent, documented group model; over-broad **exclusions** are a common hidden coverage gap.

Keep licensing and preview facts current

• Intune requires the appropriate per-user plan; Defender-for-Endpoint settings surfaced through Intune (ASR/EDR) need **Defender for Endpoint P2**; baseline versions change. **Verify current plan requirements and availability in your tenant** before relying on a feature.

Defer the policy decisions to the owners

• Which **compliance thresholds** are required, whether a **Conditional Access grant** should block or warn, and what **residual risk** is acceptable are the **security/identity team's** determinations. This solution delivers the controls and the before/after evidence — not the policy decision.

Where to go next

Once the estate is tuned and the deltas are trending the right way, natural next steps in the catalog:

• The **Microsoft Defender for Endpoint** to tune the EDR/ASR side that Intune surfaces (ASR audit→block, alert noise, web filtering).

• **Microsoft Defender XDR Deployment & Unified Onboarding** to correlate endpoint, identity, email, and cloud detections.

• The **Conditional Access / Zero Trust** identity solutions for broader access-policy design beyond compliance.

• The **Security Optimization Assessment** as the recurring read-only health check that feeds the next tuning cycle.

8. Deliverables Summary — optimization: Endpoint Security with Microsoft Intune

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box — tuning, not re-deployment

Document	Purpose
1. Solution Overview	What optimizing an Intune endpoint-security estate means — baseline tuning, conflict resolution, compliance tuning, coverage expansion; how it differs from the assessment; benefits, limitations
2. Requirements Guide	Solution-specific prerequisites (an existing, applying Intune endpoint-security deployment) with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered portal steps — baseline, migrate baseline version, resolve conflicts, tune compliance + review CA (report-only), expand coverage, validate the delta, expand
5. Testing Guide	Eight validation tests with pass/fail gates, headlined by before/after deltas
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations and where to go next

PowerShell scripts (Microsoft Graph / Intune)

Script	What it does
`SOPEIS-Discovery.ps1`	Read-only discovery — all endpoint-security and compliance policies, their assignments, applied security-baseline versions, per-policy coverage, and groups
`SOPEIS-ConflictReport.ps1`	Reports devices/settings in **Conflict** or **Error** and the policies contributing to each — the map for consolidation (the number-one drift symptom)
`SOPEIS-Validate.ps1`	Confirms the tuning applied — latest baseline on the pilot ring, conflict/error count near zero, tuned compliance policy assigned, coverage improved, and no double-targeting
`SOPEIS-Report.ps1`	Before/after metrics report — conflict/error counts, non-compliant proportion, and coverage %, with drift versus a saved baseline (proves the estate got better)
`SOPEIS-Set-Assignment.ps1`	`-WhatIf`-gated helper that adds a policy assignment to expand coverage to the pilot ring (safe preview by default)

What the buyer receives

- This solution as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-ring first, compliance/Conditional-Access changes report-only first, with a documented rollback and a before/after measurement.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A tuned Intune endpoint-security estate with **fewer conflicts** (one authoritative policy per setting, error/conflict counts down), **broader coverage** (previously unassigned devices now receiving policy), **tuned compliance** (fewer false non-compliant flags, real gaps still caught, Conditional Access impact understood), and **current baselines** — every change **piloted, measured against a captured baseline, and reversible**, so the estate is measurably better without locking users out.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SOPEIS-ConflictReport.ps1

Purpose: Read-only reporting for the "Solution Optimization — Endpoint Security with Microsoft Intune" solution. Policy conflict (one setting set by two policies) is the number-one drift symptom. This script enumerates managed devices whose configuration/compliance state is 'conflict' or 'error', and for a chosen device lists the per-setting states so you can see which policies overlap. Makes no changes. HOW TO RUN Summary across the estate (or scope to a group via -GroupName): powershell -ExecutionPolicy Bypass -File .\SOPEIS-ConflictReport.ps1 Drill into one device's per-setting states: powershell -ExecutionPolicy Bypass -File .\SOPEIS-ConflictReport.ps1 -DeviceName 'PILOT-01' Requires Microsoft.Graph.Authentication with DeviceManagementManagedDevices.Read.All and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A count of devices in conflict/error and a list of the worst offenders; with -DeviceName, the per-setting states for that device (the settings to consolidate). Compare the count to the saved baseline to prove conflicts fell. .NOTES White-label. Read-only. Reports state only; consolidation is done in the portal per Implementation Step 3. > [CmdletBinding()] param([string]$DeviceName, [string]$GroupName, [string]$OutputPath = ".\SOPEIS-conflicts.json")

How to run: Summary across the estate (or scope to a group via -GroupName): powershell -ExecutionPolicy Bypass -File .\SOPEIS-ConflictReport.ps1 Drill into one device's per-setting states: powershell -ExecutionPolicy Bypass -File .\SOPEIS-ConflictReport.ps1 -DeviceName 'PILOT-01' Requires Microsoft.Graph.Authentication with DeviceManagementManagedDevices.Read.All and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A count of devices in conflict/error and a list of the worst offenders; with -DeviceName, the per-setting states for that device (the settings to consolidate). Compare the count to the saved baseline to prove conflicts fell. .NOTES White-label. Read-only. Reports state only; consolidation is done in the portal per Implementation Step 3. > [CmdletBinding()] param([string]$DeviceName, [string]$GroupName, [string]$OutputPath = ".\SOPEIS-conflicts.json")

SOPEIS-Discovery.ps1

Purpose: Read-only discovery for the "Solution Optimization — Endpoint Security with Microsoft Intune" solution. Inventories the endpoint-security and compliance policies, their assignments, applied security-baseline versions, and per-policy assignment coverage so you can see what to tune before migrating baselines, resolving conflicts, and expanding coverage. Makes no changes. This is a TUNING engagement — it assumes Intune endpoint security is already deployed and applying. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SOPEIS-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\SOPEIS-Discovery.ps1 -OutputPath .\state.json Requires Microsoft.Graph.Authentication. Consents (read-only): DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All, Group.Read.All. EXPECTED RESULTS A before-tuning inventory: endpoint-security policies + assignments, security-baseline versions (and whether newer exist), compliance policies + assignments, and groups — the state to confirm before tuning. .NOTES White-label. Read-only. Reports configuration/identifiers only; makes no changes. Plan/baseline-version facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\SOPEIS-discovery.json")

How to run: powershell -ExecutionPolicy Bypass -File .\SOPEIS-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\SOPEIS-Discovery.ps1 -OutputPath .\state.json Requires Microsoft.Graph.Authentication. Consents (read-only): DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All, Group.Read.All. EXPECTED RESULTS A before-tuning inventory: endpoint-security policies + assignments, security-baseline versions (and whether newer exist), compliance policies + assignments, and groups — the state to confirm before tuning. .NOTES White-label. Read-only. Reports configuration/identifiers only; makes no changes. Plan/baseline-version facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\SOPEIS-discovery.json")

SOPEIS-Report.ps1

Purpose: Read-only reporting for the "Solution Optimization — Endpoint Security with Microsoft Intune" solution. Computes the key optimization metrics from Intune/Graph and — with -SaveBaseline — writes them as the 'before' snapshot. On later runs, -BaselinePath compares current metrics to the baseline so the improvement is measurable (this is what proves the estate got better). Makes no changes. HOW TO RUN Capture the BEFORE baseline (Implementation Step 1): powershell -ExecutionPolicy Bypass -File .\SOPEIS-Report.ps1 -SaveBaseline Measure the AFTER delta (Implementation Step 6): powershell -ExecutionPolicy Bypass -File .\SOPEIS-Report.ps1 -BaselinePath .\sopeis-baseline.json Requires Microsoft.Graph.Authentication with DeviceManagementManagedDevices.Read.All and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A metrics block (conflict/error count, non-compliant proportion, coverage %). With -SaveBaseline: writes sopeis-baseline.json. With -BaselinePath: prints the delta per metric. .NOTES White-label. Read-only. Coverage is share of managed devices receiving at least one endpoint-security/compliance policy in scope; refine the denominator to your estate as needed. > [CmdletBinding()] param([switch]$SaveBaseline, [string]$BaselinePath, [int]$SampleSize = 100, [string]$OutFile = ".\sopeis-baseline.json")

How to run: Capture the BEFORE baseline (Implementation Step 1): powershell -ExecutionPolicy Bypass -File .\SOPEIS-Report.ps1 -SaveBaseline Measure the AFTER delta (Implementation Step 6): powershell -ExecutionPolicy Bypass -File .\SOPEIS-Report.ps1 -BaselinePath .\sopeis-baseline.json Requires Microsoft.Graph.Authentication with DeviceManagementManagedDevices.Read.All and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A metrics block (conflict/error count, non-compliant proportion, coverage %). With -SaveBaseline: writes sopeis-baseline.json. With -BaselinePath: prints the delta per metric. .NOTES White-label. Read-only. Coverage is share of managed devices receiving at least one endpoint-security/compliance policy in scope; refine the denominator to your estate as needed. > [CmdletBinding()] param([switch]$SaveBaseline, [string]$BaselinePath, [int]$SampleSize = 100, [string]$OutFile = ".\sopeis-baseline.json")

SOPEIS-Set-Assignment.ps1

Purpose: Change helper for the "Solution Optimization — Endpoint Security with Microsoft Intune" solution. Adds an include assignment for the named group (e.g. the pilot ring) to a named settings-catalog / endpoint-security policy so previously unassigned devices receive it. This is the only script that can change configuration, and it DEFAULTS TO A SAFE PREVIEW. Expanding coverage must not double-target a device (which re-creates conflict) — validate with SOPEIS-ConflictReport.ps1 after applying. HOW TO RUN Preview the assignment that WOULD be added (default — changes nothing): powershell -ExecutionPolicy Bypass -File .\SOPEIS-Set-Assignment.ps1 `-PolicyName 'Endpoint Security - Antivirus - Prod' -GroupName 'Intune Optimize - Pilot Ring - Pilot'` Apply (explicit — required to make the change): powershell -ExecutionPolicy Bypass

`-File .\SOPEIS-Set-Assignment.ps1` -PolicyName 'Endpoint Security - Antivirus - Prod' -GroupName 'Intune Optimize - Pilot Ring - Pilot' -Apply Requires Microsoft.Graph with DeviceManagementConfiguration.ReadWrite.All and Group.Read.All. EXPECTED RESULTS Preview: prints the resolved policy id, group id, and the include assignment it WOULD add (preserving existing assignments). Apply: adds the assignment; validate coverage with SOPEIS-Validate.ps1 and re-check for double-targeting with SOPEIS-ConflictReport.ps1. .NOTES White-label. DEFAULTS TO -WhatIf/preview. Pilot ring first; do not double-target a device. Rollback: remove the added assignment in the portal or re-apply the prior assignment set. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$PolicyName, [Parameter(Mandatory)] [string]$GroupName, [switch]$Apply)

How to run: Preview the assignment that WOULD be added (default — changes nothing): powershell -ExecutionPolicy Bypass -File .\SOPEIS-Set-Assignment.ps1 `-PolicyName 'Endpoint Security - Antivirus - Prod' -GroupName 'Intune Optimize - Pilot Ring - Pilot' Apply (explicit — required to make the change): powershell -ExecutionPolicy Bypass -File .\SOPEIS-Set-Assignment.ps1` -PolicyName 'Endpoint Security - Antivirus - Prod' -GroupName 'Intune Optimize - Pilot Ring - Pilot' -Apply Requires Microsoft.Graph with DeviceManagementConfiguration.ReadWrite.All and Group.Read.All. EXPECTED RESULTS Preview: prints the resolved policy id, group id, and the include assignment it WOULD add (preserving existing assignments). Apply: adds the assignment; validate coverage with SOPEIS-Validate.ps1 and re-check for double-targeting with SOPEIS-ConflictReport.ps1. .NOTES White-label. DEFAULTS TO -WhatIf/preview. Pilot ring first; do not double-target a device. Rollback: remove the added assignment in the portal or re-apply the prior assignment set. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$PolicyName, [Parameter(Mandatory)] [string]$GroupName, [switch]$Apply)

SOPEIS-Validate.ps1

Purpose: Read-only validation for the "Solution Optimization — Endpoint Security with Microsoft Intune" solution. Checks the pass/fail gates from the Testing Guide and reports PASS/WARN per check with an overall verdict. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SOPEIS-Validate.ps1 powershell -ExecutionPolicy Bypass -File .\SOPEIS-Validate.ps1 `-BaselineName 'Security Baseline - Pilot Ring - Pilot'` -ComplianceName 'Compliance - Windows Corporate - Prod' ` -PilotGroupName 'Intune Optimize - Pilot Ring - Pilot' Requires Microsoft.Graph.Authentication with read scopes (DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All, Group.Read.All). EXPECTED RESULTS A PASS/WARN line per check and an overall verdict — the gate for expanding beyond the pilot ring (Implementation Step 7). .NOTES White-label. Read-only. Baseline-version currency and Conditional Access enforce state are confirmed in the portal per the Testing Guide. > [CmdletBinding()] param([string]$BaselineName = 'Security Baseline - Pilot Ring - Pilot', [string]$ComplianceName = 'Compliance - Windows Corporate - Prod', [string]$PilotGroupName = 'Intune Optimize - Pilot Ring - Pilot')

How to run: powershell -ExecutionPolicy Bypass -File .\SOPEIS-Validate.ps1 powershell -ExecutionPolicy Bypass -File .\SOPEIS-Validate.ps1 `-BaselineName 'Security Baseline - Pilot Ring - Pilot'` -ComplianceName 'Compliance - Windows Corporate - Prod' ` -PilotGroupName 'Intune Optimize - Pilot Ring - Pilot' Requires Microsoft.Graph.Authentication with read scopes (DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All, Group.Read.All). EXPECTED RESULTS A PASS/WARN line per check and an overall verdict — the gate for expanding beyond the pilot ring (Implementation Step 7). .NOTES White-label. Read-only. Baseline-version currency and Conditional Access enforce state are confirmed in the portal per the Testing Guide. > [CmdletBinding()] param([string]$BaselineName = 'Security Baseline - Pilot Ring - Pilot',

[string]$ComplianceName = 'Compliance - Windows Corporate - Prod', [string]$PilotGroupName = 'Intune Optimize - Pilot Ring - Pilot')